

Министерство науки и высшего образования Российской Федерации
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ЯДЕРНЫЙ УНИВЕРСИТЕТ
«МИФИ»

ИНСТИТУТ ЛАЗЕРНЫХ И ПЛАЗМЕННЫХ ТЕХНОЛОГИЙ
Кафедра прикладной математики №31

ОТЧЕТ
по научно-исследовательской работе за 6 семестр
на тему:

Автоматизация аналитического исследования нелинейных
дифференциальных уравнений

Группа
Студент
Научный руководитель
старший преподаватель каф. №31

Б23-215
А.К. Кораблёва
А.А. Кутуков

(подпись)

Москва 2026

Аннотация

В работе исследуется проблема трудоёмкости аналитического исследования нелинейных дифференциальных уравнений (НДУ), широко применяемых в математической физике и теории волн. Целью является разработка автоматизированного вычислительного инструментария, объединяющего сингулярный анализ и построение точных решений. В качестве платформы выбрана среда Python с библиотекой символьных вычислений SymPy. Были разработаны два модуля: первый реализует решатель уравнения Курамото–Сивашинского на основе анзацев Риккати и эллиптической функции Вейерштрасса, второй — метод доминантного баланса для автоматического определения порядка и типа подвижной особенности (полюс, точка ветвления, логарифмическая особенность и др.). Алгоритм поиска порядка полюса не связан напрямую с программой решения уравнения Курамото–Сивашинского и функционирует как универсальный диагностический инструмент. Его корректность верифицирована на обширном тестовом наборе, включающем классические уравнения математической физики (уравнения Пенлеве I–IV, Кортевега–де Фриза, Бюргерса, Фишера–Колмогорова и др.), где продемонстрирована 100% точность согласования с аналитическими эталонами. Интеграция модулей в единый интерфейс существенно сокращает трудоёмкость ручных выкладок и расширяет возможности аналитического исследования широкого класса нелинейных динамических систем.

Содержание

1.	Введение	5
2.	Уравнение Курамото-Сивашинского	6
2.1.	Математическая модель и редукция к ОДУ	6
2.2.	Решение аналитическими методами: анзацы Риккати и Вейерштрасса	8
2.2.1	Определение порядка полюса уравнения Курамото-Сивашинского	8
2.2.2.	Аналитическое решение методом Риккати	8
2.2.3.	Аналитическое решение методом Вейерштрасса	9
2.3.	Алгоритмизация символьных вычислений и переход к Python/SymPy	11
2.4	Визуализация и качественный анализ волновых профилей	12
2.4.1	Построение графиков решений	12
2.4.2	Анализ переходов между режимами	15
2.6.	Верификация результатов и обсуждение вычислительной эффективности	16
3.	Поиск порядка сингулярности	17
3.1	Введение в проблему сингулярностей	17
3.1.1.	Определение сингулярной точки решения ОДУ	17
3.1.2	Классификация особенностей	17
3.1.3	Польза информации о порядке полюса	18
3.2	Теоретические основы метода доминантного баланса	18
3.2.1.	Идея метода: баланс наиболее сингулярных слагаемых	18
3.2.2.	Алгебраический анзац	19
3.2.3	Вычисление показателей для производных: падающий факториал	19
3.2.4	Нелинейные и смешанные мономы: аддитивность показателей	19
3.2.5	Рациональная нелинейность $P(y)/Q(y)$	20
3.2.6	Логарифмический анзац для уравнений с $e^{\alpha y}$	20
3.2.7	Обработка трансцендентных функций	20
3.3	Алгоритмическая реализация (разбор кода <code>find_pole_order.py</code>)	21
3.3.1	Низкоуровневые операции с показателями ξ	21
3.3.2	Классификация трансцендентных функций	21
3.3.3.	Обработка переменных коэффициентов	21
3.3.4.	Подстановка анзаца	22
3.3.5	Баланс показателей	22
3.3.6	Классификация результата	22
3.3.7	Публичный API	23

3.4	Верификация и тестирование	23
3.4.1	Набор тестов	23
3.4.2	Критерии успеха	23
3.4.3	Граничные случаи	24
3.5	Ограничения метода и направления развития	24
3.5.1	Ограничения метода	24
3.5.2	Возможные расширения	24
3.5.3	Интеграция с другими методами	24
3.5.4	Краткое резюме	25
4.	Заключение	26
5.	Список литературы и интернет-ресурсов	27
	Приложение А. Доступ к исходным материалам	28

1. Введение

Нелинейные дифференциальные уравнения (НДУ) составляют математический фундамент описания сложных физических, биологических и инженерных процессов: от турбулентности и горения до нелинейной оптики и динамики популяций. В отличие от линейных моделей, аналитические решения НДУ встречаются редко, однако их наличие критически важно для понимания структуры фазового пространства, анализа устойчивости, верификации численных схем и конструирования управляемых волновых структур. Традиционные методы поиска точных решений — анализ Пенлеве, метод доминантного баланса, анзацы Риккати и эллиптической функции Вейерштрасса — обладают высокой эффективностью, но их ручная реализация сопряжена с громоздкими символьными выкладками, высокой вероятностью вычислительных ошибок и требует глубокой математической подготовки.

В современных условиях наблюдается острая потребность в автоматизации символьных вычислений. Коммерческие компьютерные алгебраические системы, такие как Maple, традиционно используются для прототипирования и проверки корректности аналитических алгоритмов благодаря мощным встроенным средствам символьной математики. Однако их коммерческая модель распространения, закрытая лицензионная политика и ограничения на интеграцию с открытыми научными инструментами обуславливают необходимость переноса алгоритмов в свободную экосистему Python. Результатом данной работы стал набор программных модулей, обеспечивающих автоматизацию сингулярного анализа и построения точных решений для уравнения Курамото-Сивашинского.

Целью настоящей научно-исследовательской работы является алгоритмизация и программная реализация методов аналитического исследования нелинейных дифференциальных уравнений на базе открытых технологий. Для достижения поставленной цели решаются следующие задачи:

1. Изучить теоретические основы сингулярного анализа, метод доминантного баланса и классификацию особенностей решений.
2. Разработать теоретический аппарат аналитического решения уравнения Курамото–Сивашинского и реализовать его поиск двумя методами (Риккати и Вейерштрасса) в среде Maple.
3. Перенести верифицированные алгоритмы в среду Python с использованием библиотеки SymPy, создав отдельный программный модуль для поиска точных решений уравнения Курамото–Сивашинского.
4. Разработать программный модуль для автоматического определения порядка и типа подвижной особенности (полюса, точки ветвления) одиночных ОДУ и систем уравнений.
5. Сформировать верификационный набор тестов для модуля поиска полюсов и провести отладку на классических уравнениях математической физики.

Научная новизна работы заключается в переносе и адаптации классических методов сингулярного анализа в открытую среду Python. Разработанный алгоритм доминантного баланса корректно обрабатывает смешанные мономы, дробно-рациональные нелинейности, логарифмические анзацы и переменные коэффициенты, что расширяет область применимости традиционных подходов. Программная реализация выполнена на языке Python, что гарантирует кроссплатформенность, открытый исходный код и лёгкую интеграцию в современные научные проекты.

Практическая значимость работы обусловлена возможностью использования разработанных инструментов в исследовательской деятельности для быстрой диагностики сингулярностей новых нелинейных моделей, а также в учебном процессе при изучении методов математической физики и компьютерной алгебры. Автоматизация рутинных символьных операций позволяет исследователям сосредоточиться на физической интерпретации результатов и постановке новых задач.

Структура отчёта выстроена в соответствии с логикой разработки двух независимых программных модулей. В первой части рассматривается уравнение Курамото–Сивашинского: приводятся аналитические выкладки для методов Риккати и Вейерштрасса, демонстрируется поиск и визуализация точных решений в среде Maple, а также представлена полная перенесённая в Python/SymPy реализация с интерактивным меню для работы с полученными семействами решений. Во второй части излагаются теоретические основы метода доминантного баланса и классификации особенностей, описывается архитектура модуля поиска порядка полюса (от прототипа до самостоятельного инструмента на Python), приводятся результаты автоматизированного тестирования на наборе классических уравнений математической физики и анализируется корректность работы алгоритма. Заключительная часть содержит выводы по выполненной работе и перспективы дальнейшего развития предложенного вычислительного комплекса.

2. Уравнение Курамото–Сивашинского

2.1. Математическая модель и редукция к ОДУ

В основе предлагаемого алгоритмического подхода лежит метод простейших уравнений, подробно изложенный в монографии Н. А. Кудряшова «Методы нелинейной математической физики». Данный метод представляет собой универсальный конструктивный аппарат для поиска точных решений нелинейных дифференциальных уравнений. Его основная идея заключается в замене исходной неизвестной функции на решение некоторого вспомогательного дифференциального уравнения более низкого порядка, которое допускает аналитическое интегрирование (так называемого простейшего уравнения). В качестве простейших уравнений чаще всего выступают уравнение Риккати $Y' = b - Y^2$, уравнение для эллиптической функции Вейерштрасса $(R')^2 = -4R^3 + aR^2 + 2bR + c$, логистическое уравнение или линейные уравнения с постоянными коэффициентами. Подстановка такого анзаца преобразует исходное нелинейное ОДУ в полиномиальную систему алгебраических уравнений относительно неизвестных коэффициентов разложения и параметров уравнения.

Объектом исследования в данной работе выступает уравнение Курамото–Сивашинского:

$$u_t + uu_x + u_{xx} + \sigma u_{xxx} + u_{xxxx} = 0, \quad (1)$$

которое является одной из фундаментальных моделей нелинейной математической физики. Уравнение описывает динамику фронтов пламени, течение тонких плёнок жидкости на наклонных плоскостях, а также процессы в реакционно-диффузионных системах и плазме. Физико-математическая структура уравнения (1) отражает конкуренцию нескольких фундаментальных механизмов:

- Нелинейность (uu_x) отвечает за конвективный перенос и формирование крутых фронтов (эффект обострения);
- Диссипация/Антидиссипация (u_{xx}) в данном контексте играет дестабилизирующую роль, приводя к длинноволновой неустойчивости;
- Дисперсия (σu_{xxx}) отвечает за зависимость фазовой скорости волны от её длины, способствуя осцилляциям профиля;
- Высшая диссипация (u_{xxxx}) обеспечивает стабилизацию системы на малых масштабах, предотвращая коллапс решений и ограничивая спектр неустойчивых мод.

Взаимодействие этих эффектов порождает богатую динамику, включая хаотические режимы и образование устойчивых когерентных волновых структур.

Для поиска точных решений уравнение (1) редуцируется к обыкновенному дифференциальному уравнению с помощью стандартного анзаца бегущей волны:

$$u(x, t) = y(z), \quad z = x - C_0 t, \quad (2)$$

где C_0 — скорость распространения волны. Подставляя производные

$$u_t = -C_0 y', \quad u_x = y', \quad u_{xx} = y'', \quad u_{xxx} = y''', \quad u_{xxxx} = y'''' ,$$

в уравнение (1), получаем нелинейное ОДУ четвёртого порядка:

$$y'''' + \sigma y''' + y'' + yy' - C_0 y' = 0. \quad (3)$$

Интегрируя это уравнение один раз по переменной z и вводя постоянную интегрирования C_1 , приходим к автономному ОДУ третьего порядка:

$$y''' + \sigma y'' + y' + \frac{1}{2}y^2 - C_0 y + C_1 = 0. \quad (4)$$

Именно уравнение (4) является основным объектом дальнейшей аналитической и компьютерной обработки.

Параметры уравнения (4) играют ключевую роль в формировании волновых профилей и определяют условия существования точных решений:

- Параметр C_0 определяет скорость бегущей волны в лабораторной системе координат. Его значение напрямую влияет на баланс между нелинейным членом $\frac{1}{2}y^2$ и линейным членом $-C_0 y$, определяя амплитуду и асимптотическое поведение решения при $z \rightarrow \pm\infty$.
- Параметр σ характеризует интенсивность дисперсионных эффектов. При $\sigma = 0$ уравнение переходит в классическую форму Курамото–Сивашинского. Отличие σ от нуля модифицирует фазовую структуру решений, приводя к появлению асимметричных профилей и изменению условий существования солитонов.
- Параметр C_1 является константой интегрирования, возникающей при редукции к ОДУ. Он задаёт фоновое значение решения (средний уровень волны) и сдвигает потенциальную яму в соответствующей динамической системе, влияя на тип сепаратрис и, следовательно, на качественную форму получаемых точных решений (солитоны, кноидальные волны, рациональные решения).

Совокупность этих параметров определяет область существования точных решений, что делает задачу их аналитического нахождения чувствительной к выбору начальных алгебраических условий, решаемую в рамках автоматизированной реализации метода простейших уравнений.

2.2. Решение аналитическими методами: анзацы Риккати и Вейерштрасса

После редукции уравнения Курамото–Сивашинского к автономному ОДУ третьего порядка

$$y''' + \sigma y'' + y' + \frac{1}{2}y^2 - C_0 y + C_1 = 0, \quad (5)$$

возникает задача построения его точных решений. Поскольку уравнение содержит полиномиальную нелинейность (в данном случае квадратичную y^2), эффективным инструментом является метод простейших уравнений. Степень полинома в анзаце определяется не степенью нелинейности, а порядком подвижного полюса решения p , что гарантирует конечность разложения и позволяет свести задачу к решению системы алгебраических уравнений.

2.2.1 Определение порядка полюса уравнения Курамото–Сивашинского

Порядок полюса p находится методом доминантного баланса. В окрестности подвижной особенности полагаем асимптотику $y(z) \sim A\xi^{-p}$, где $\xi = z - z_0 \rightarrow 0$. Доминирующими слагаемыми в (5) являются старшая производная y''' и нелинейный член $\frac{1}{2}y^2$. Их ведущие показатели:

$$y''' \sim \xi^{-p-3}, \quad \frac{1}{2}y^2 \sim \xi^{-2p}. \quad (6)$$

Условие баланса $\xi^{-p-3} \sim \xi^{-2p}$ даёт алгебраическое уравнение

$$-p - 3 = -2p \quad \Rightarrow \quad p = 3. \quad (7)$$

Поскольку $p \in \mathbb{Z}_{>0}$, особенность является полюсом третьего порядка. Это обосновывает использование полиномиальных анзацев третьей степени по решениям простейших уравнений.

2.2.2. Аналитическое решение методом Риккати

В качестве простейшего уравнения выбираем уравнение Риккати:

$$Y' = b - Y^2, \quad b = \text{const}. \quad (8)$$

Его общее решение при $b > 0$ имеет вид $Y(z) = \sqrt{b} \tanh(\sqrt{b}(z + z_0))$. С учётом $p = 3$ ищем решение уравнения (4) в виде кубического полинома:

$$y(z) = A_0 + A_1 Y + A_2 Y^2 + A_3 Y^3. \quad (9)$$

Вычисляем производные y , последовательно используя правило замены $d/dz = (b - Y^2)d/dY$:

$$y' = -3A_3 Y^4 - 2A_2 Y^3 + (3A_3 b - A_1) Y^2 + 2A_2 b Y + A_1 b, \quad (10)$$

$$y'' = 12A_3 Y^5 + 10A_2 Y^4 - 12A_3 b Y^3 - 6A_2 b Y^2 + (3A_3 b^2 - 3A_1 b) Y + 2A_2 b^2, \quad (11)$$

$$y''' = -60A_3 Y^6 - 50A_2 Y^5 + (60A_3 b - 6A_1) Y^4 + 20A_2 b Y^3 + \dots \quad (12)$$

Подстановка y, y', y'', y''' в уравнение (4) и раскрытие скобок приводит к полиному относительно Y . Приравнявая коэффициенты при каждой степени Y^k ($k = 0, \dots, 6$) к нулю, получаем переопределённую систему алгебраических уравнений относительно неизвестных $A_0, A_1, A_2, A_3, b, \sigma, C_0, C_1$.

Решение системы показывает, что $A_3 = 120$, $A_2 = -15\sigma$, а параметры b и σ связаны дискретными соотношениями. В зависимости от значения σ получаем следующие семейства точных решений:

1. Случай $\sigma = \pm 4$, $b = 1/4$:

$$y(z) = C_0 \mp 9 - 30Y + 60(\pm 1)Y^2 + 120Y^3, \quad Y(z) = \frac{1}{2} \tanh\left(\frac{z + z_0}{2}\right). \quad (13)$$

Данные решения описывают уединённые волны (солитоны) с гладким переходом между асимптотиками $y(\pm\infty) = C_0 \mp 9$.

2. Случай $\sigma = 0$, $b = 11/76$:

$$y(z) = C_0 - \frac{270}{19}Y + 120Y^3, \quad Y(z) = \sqrt{\frac{11}{76}} \tanh\left(\sqrt{\frac{11}{76}}(z + z_0)\right). \quad (14)$$

3. Случай $\sigma = \pm \frac{12\sqrt{47}}{47}$, $b = 1/188$:

$$y(z) = C_0 \mp \frac{45\sqrt{47}}{2209} + \frac{90}{47}Y \pm \frac{180\sqrt{47}}{47}Y^2 + 120Y^3, \quad Y(z) = \frac{1}{2\sqrt{47}} \tanh\left(\frac{z + z_0}{2\sqrt{47}}\right). \quad (15)$$

4. Случай $\sigma = \pm \frac{16\sqrt{73}}{73}$, $b = 1/292$:

$$y(z) = C_0 \mp \frac{60\sqrt{73}}{5329} + \frac{150}{73}Y \pm \frac{240\sqrt{73}}{73}Y^2 + 120Y^3, \quad Y(z) = \frac{1}{2\sqrt{73}} \tanh\left(\frac{z + z_0}{2\sqrt{73}}\right). \quad (16)$$

2.2.3. Аналитическое решение методом Вейерштрасса

Для построения более общих эллиптических решений используем анзац, содержащий как саму функцию $R(z)$, так и её производную $R'(z)$:

$$y(z) = A_0 + A_1 R(z) + A_2 R'(z). \quad (17)$$

В качестве определяющего уравнения для $R(z)$ выбираем уравнение первого рода для эллиптической функции Вейерштрасса:

$$(R')^2 = -4R^3 + aR^2 + 2bR + c. \quad (18)$$

Дифференцируя соотношение (18) по z , выражаем старшие производные R через базисные функции R и R' :

$$\begin{aligned} R'' &= -6R^2 + aR + b, \\ R''' &= -12RR' + aR', \\ R^{(4)} &= -12(R')^2 - 12RR'' + aR''. \end{aligned}$$

Вычислим производные функции $y(z)$ из анзаца (17):

$$\begin{aligned}y' &= A_1 R' + A_2 R'', \\y'' &= A_1 R'' + A_2 R''', \\y''' &= A_1 R''' + A_2 R^{(4)}.\end{aligned}$$

Подставляя выражения для $R'', R''', R^{(4)}$ и используя уравнение (18) для исключения $(R')^2$, все производные y', y'', y''' могут быть представлены в виде полиномов от переменных R и R' .

Подставим y, y', y'', y''' в уравнение Курамото–Сивашинского (4). После раскрытия скобок и приведения подобных слагаемых, левая часть уравнения становится полиномом по базису $\{1, R, R^2, R^3, R', RR', (R')^2\}$. Используя (18) для замены $(R')^2$ на полином третьей степени от R , приводим выражение к полиному от R и R' .

Условие обращения этого полинома в тождественный ноль эквивалентно системе алгебраических уравнений на коэффициенты A_i и параметры уравнения. Приравнявая коэффициенты при старших степенях R (в частности, при R^3 и R^2) к нулю, определяем ведущие коэффициенты анзаца:

$$A_2 = 60, \quad A_1 = 15\sigma. \quad (19)$$

Далее, приравнявая коэффициенты при R', RR', R^2, R и свободный член, находим связь между параметрами уравнения Вейерштрасса и параметрами уравнения Курамото–Сивашинского:

$$\sigma = \pm 4, \quad b = \frac{1}{24} - \frac{a^2}{24}. \quad (20)$$

Параметр C определяется из условия баланса при R^0 :

$$c = \frac{C_0^2}{2160} - \frac{C_1}{1080} + \frac{a^3}{432} - \frac{a}{144} - \frac{13}{1080}. \quad (21)$$

Таким образом, получаем, что уравнение Курамото–Сивашинского при $\sigma = \pm 4$ имеет решение в виде

$$y(z) = C_0 \mp 5a \mp 1 \pm 60R + 60R_z, \quad (22)$$

где $R(z)$ является решением уравнения (18) с подставленными найденными параметрами.

Для явного вида решения необходимо выразить функцию $R(z)$ через эллиптические функции Якоби. Рассмотрим кубический многочлен в правой части (18):

$$-4R^3 + aR^2 + 2bR + c = -4(R - R_1)(R - R_2)(R - R_3) = 0, \quad (23)$$

где R_1, R_2, R_3 — действительные корни, упорядоченные как $R_1 \geq R_2 \geq R_3$.

В этом случае периодическое решение уравнения (18) записывается через квадрат эллиптического косинуса cn :

$$R(z) = R_2 + (R_1 - R_2) \text{cn}^2 \left(\sqrt{R_1 - R_3} z, S \right), \quad (24)$$

где S — модуль эллиптической функции, определяемый соотношением:

$$S^2 = \frac{R_1 - R_2}{R_1 - R_3}. \quad (25)$$

Для подстановки в анзац (17) требуется также производная $R'(z)$. Дифференцируя выражение для $R(z)$, получаем:

$$R'(z) = -2(R_1 - R_2)\sqrt{R_1 - R_3} \operatorname{cn}(\chi, S) \operatorname{sn}(\chi, S) \operatorname{dn}(\chi, S), \quad (26)$$

где $\chi = \sqrt{R_1 - R_3} z$, а sn и dn — эллиптические синус и дельта-амплитуда соответственно.

Таким образом, общее решение уравнения Курамото–Сивашинского методом Вейерштрасса принимает вид:

$$y(z) = A_0 + A_1 [R_2 + (R_1 - R_2) \operatorname{cn}^2(\chi, S)] - 2A_2(R_1 - R_2)\sqrt{R_1 - R_3} \operatorname{cn} \operatorname{sn} \operatorname{dn}. \quad (27)$$

Параметр модуля S определяет форму волны:

- При $S \rightarrow 0$ (когда $R_1 \rightarrow R_2$) эллиптический косинус переходит в обычный тригонометрический косинус: $\operatorname{cn}(u, 0) = \cos u$. Решение описывает гармонические колебания.
- При $S \rightarrow 1$ (когда $R_2 \rightarrow R_3$) функция $\operatorname{cn}(u, 1)$ вырождается в гиперболический секанс: $\operatorname{cn}(u, 1) = \operatorname{sech} u = 1/\cosh u$. В этом пределе период волны стремится к бесконечности, и мы получаем уединённую волну (солитон).

2.3. Алгоритмизация символьных вычислений и переход к Python/SymPy

На начальном этапе исследования верификация гипотез и поиск коэффициентов выполнялись в среде Maple. Процесс включал ручную подстановку анзаца в дифференциальное уравнение, раскрытие скобок и приведение подобных слагаемых. Основная трудность заключалась в автоматизации сбора коэффициентов при степенях вспомогательной функции $Y(z)$ или $R(z)$, что требовало значительных временных затрат и ручного контроля при изменении структуры уравнения или порядка анзаца.

Для создания универсального и воспроизводимого инструмента алгоритм был перенесен в среду Python с использованием библиотеки символьной математики SymPy. Архитектура разработанного решателя (функция `solve_riccati_symbolic`) построена на принципах модульности и разделяет процесс вычисления на несколько независимых функциональных блоков:

1. Формирование анзаца и производных. Функция `construct_ansatz` генерирует полиномиальное выражение для $y(z)$ заданной степени p . Для корректного вычисления производных используется функция `compute_derivatives`, которая применяет правило цепной дифференциации с учетом определяющего уравнения Риккати ($Y' = b - Y^2$). Производные высших порядков вычисляются рекурсивно через замену $d/dz = (b - Y^2)d/dY$, что позволяет избежать громоздких выражений в терминах z и работать исключительно в базисе Y .

2. Подстановка и приведение уравнения. Основная процедура `substitute_ode` подставляет полученный анзац и его производные в уравнение Курамото–Сива-шинского. После подстановки применяется метод `sp.expand` для полного раскрытия скобок и приведения выражения к полиномиальному виду относительно вспомогательной переменной Y .
3. Извлечение алгебраической системы. Функция `extract_coefficients` анализирует полученный полином. Используя метод `sp.coeff`, она последовательно извлекает коэффициенты при каждой степени Y^k (от $k = 0$ до $k = 2p$) и приравняет их к нулю. Это формирует переопределенную систему алгебраических уравнений относительно неизвестных параметров A_i , b , σ и констант интегрирования C_0, C_1 .
4. Решение и фильтрация результатов. Система передается в функцию `sp.solve`. Поскольку система переопределена, решатель автоматически проверяет условия совместимости. В результате нахождения решений формируются зависимости между параметрами (например, дискретные значения $\sigma = \pm 4$). Для повышения эффективности реализован блок постобработки `filter_solutions`, который:
 - Отсеивает тривиальные решения (например, $y = 0$).
 - Объединяет дубликаты, возникающие при различных путях символьных преобразований.
 - Группирует семейства решений по типу нелинейности (солитоны, периодические волны).

Важным аспектом реализации стала интеграция с библиотекой визуализации `matplotlib`. Функция `plot_solution` принимает найденные символьные коэффициенты и строит графики волновых профилей, позволяя пользователю интерактивно исследовать влияние параметров σ и b на форму волны. Интерфейс взаимодействия с пользователем реализован в виде консольного меню, обеспечивающего пошаговый доступ ко всем этапам: от выбора типа анзаца до построения графика найденного решения.

2.4 Визуализация и качественный анализ волновых профилей

В данном разделе проводится численная визуализация найденных аналитических решений уравнения Курамото–Сивашинского и качественный анализ их поведения при варьировании управляющих параметров.

2.4.1 Построение графиков решений

Для визуализации решений, полученных методами Риккати и Вейерштрасса, использовалась библиотека `matplotlib` в среде Python. Все графики строятся для переменной бегущей волны $z = x - C_0 t$, что позволяет анализировать стационарные профили в движущейся системе координат.

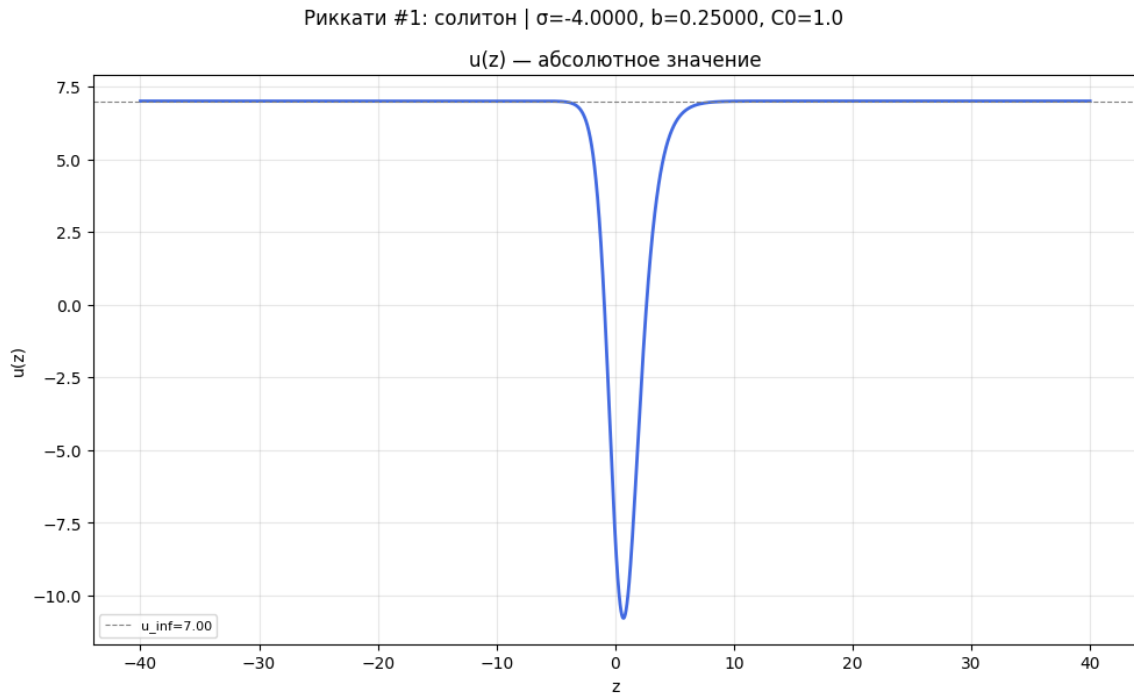


Рис. 1. Локализованный солитонный профиль решения Риккати при $\sigma = -4$, $b = 1/4$, $C_0 = 1$. Видна характерная асимптотика $y(z) \rightarrow u_\infty$ при $|z| \rightarrow \infty$.

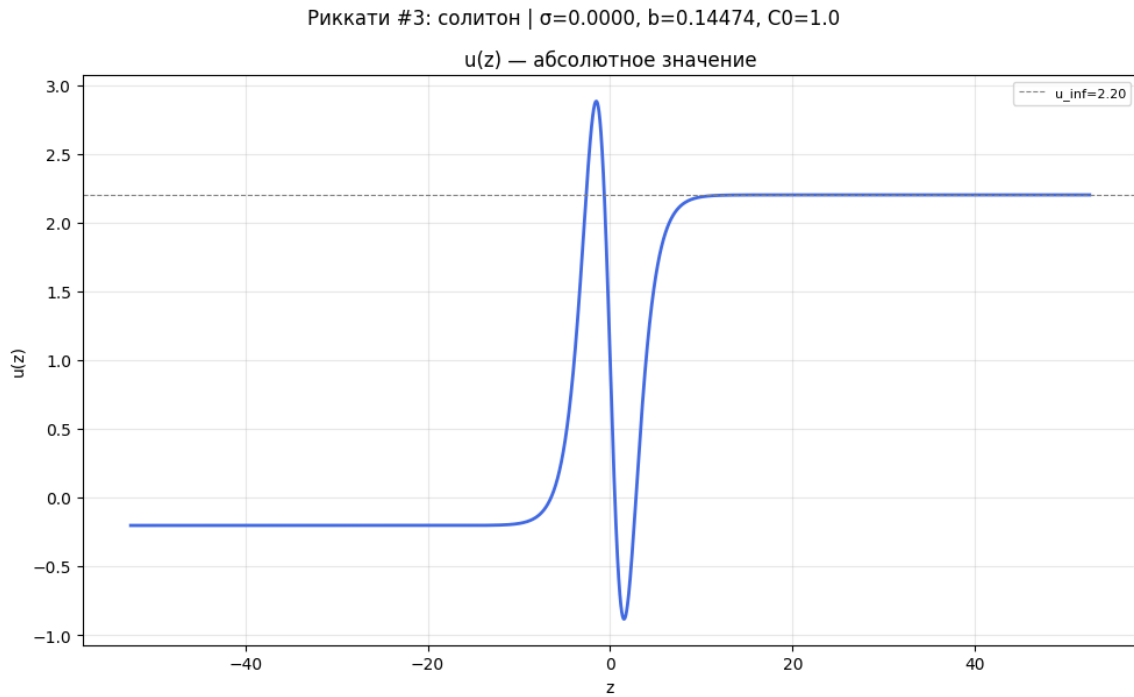


Рис. 2. Локализованный солитонный профиль решения Риккати при $\sigma = 0$, $b \approx 0.145$, $C_0 = 1$.

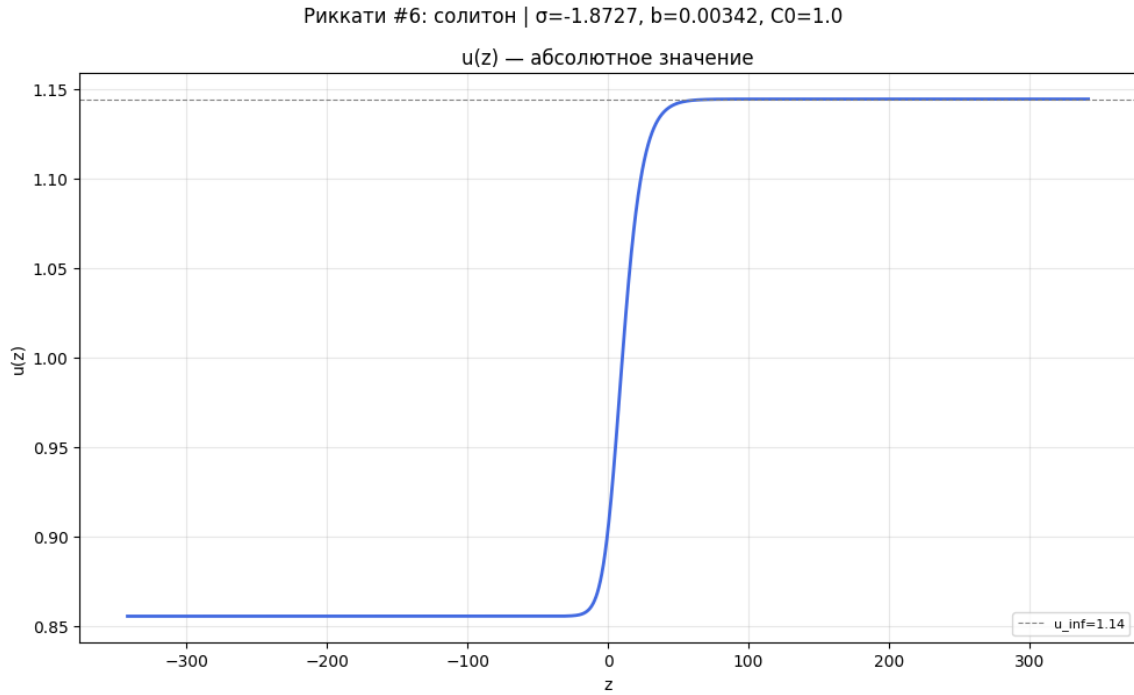


Рис. 3. Локализованный солитонный профиль решения Риккати при $\sigma = -\frac{16\sqrt{73}}{73}$, $b \approx 0.034$, $C_0 = 1$.

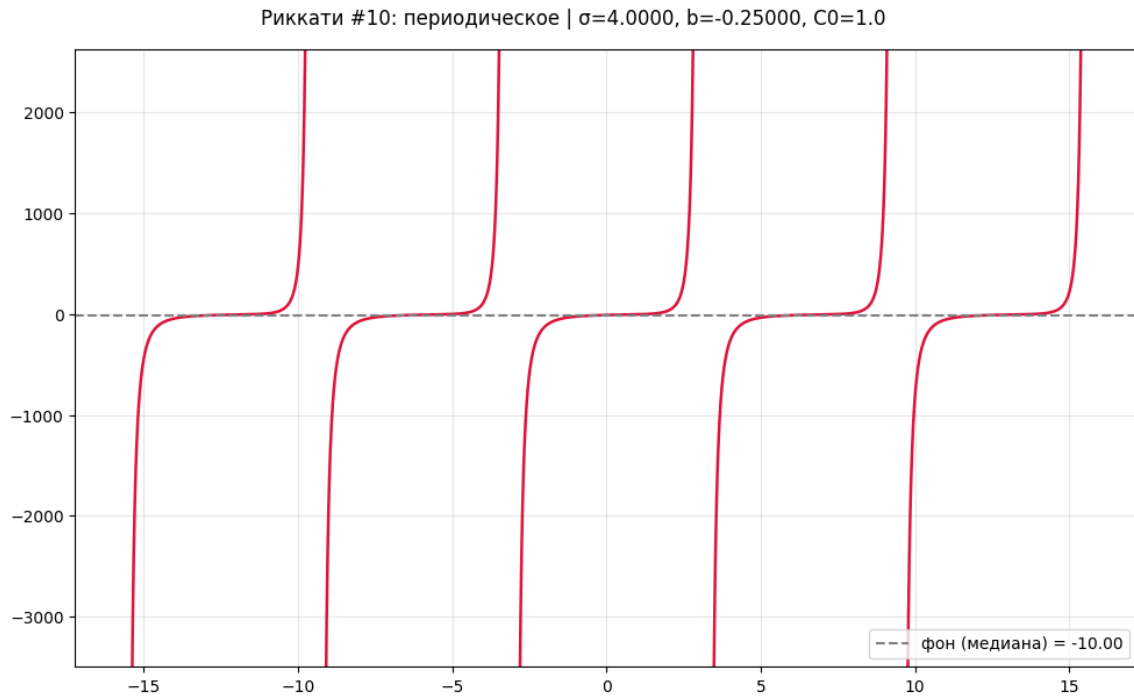


Рис. 4. Периодическая структура решения Риккати при $\sigma = 4$, $b = -1/4$. Наблюдаются полюсные особенности, соответствующие точкам, где $\tanh(kz)$ заменяется на $\tan(kz)$.

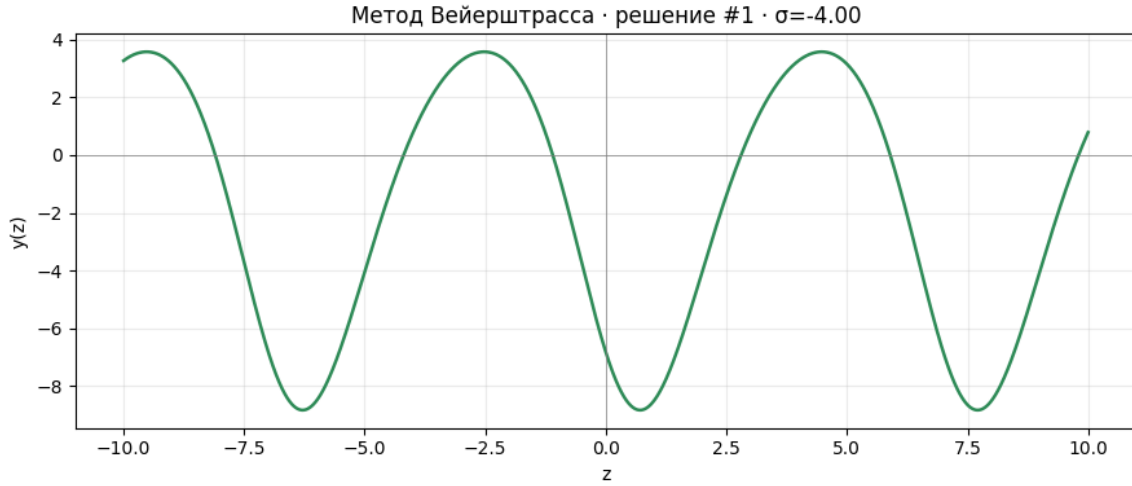


Рис. 5. Эллиптическое решение метода Вейерштрасса при $\sigma = -4$, $a = 1$, $b = 0$, $c \approx -0.004$. Профиль описывается функцией $\wp(z; g_2, g_3)$ с тремя вещественными корнями кубического полинома.

2.4.2 Анализ переходов между режимами

Ключевым параметром, определяющим тип волновой структуры, является знак константы b в вспомогательном уравнении Риккати $Y' = b - Y^2$.

Случай $b > 0$: локализованные и однородные волны. Общее решение уравнения Риккати имеет вид

$$Y(z) = \sqrt{b} \tanh(\sqrt{b}(z - z_0)), \quad (28)$$

что приводит к локализованным солитонным профилям для $y(z)$. Характерные свойства:

- Экспоненциальная сходимость к постоянному фону: $y(z) \rightarrow u_\infty$ при $|z| \rightarrow \infty$;
- Конечная ширина фронта, пропорциональная $b^{-1/2}$;
- Отсутствие особенностей на вещественной оси.

Такие решения соответствуют уединённым волнам, наблюдаемым в диссипативных системах с балансом нелинейности и дисперсии.

Случай $b < 0$: периодические и полюсные структуры. Решение уравнения Риккати выражается через тангенс:

$$Y(z) = \sqrt{|b|} \tan(\sqrt{|b|}(z - z_0)), \quad (29)$$

что порождает периодические структуры с полюсами:

- Период $T = \pi/\sqrt{|b|}$;
- Полюсы первого порядка в точках $z_n = z_0 + \frac{\pi}{2\sqrt{|b|}} + nT$, $n \in \mathbb{Z}$;
- Расходимость амплитуды в окрестности полюсов: $y(z) \sim (z - z_n)^{-k}$, где k определяется порядком анзаца.

Такие решения характерны для режимов с сильной неустойчивостью и могут описывать фронты разрушения или зоны интенсивной генерации.

2.6. Верификация результатов и обсуждение вычислительной эффективности

Для подтверждения корректности разработанного алгоритма была проведена кросс-верификация с эталонными вычислениями, выполненными на этапе прототипирования в среде Maple. Символьные выражения для коэффициентов анзацев Риккати и Вейерштрасса, полученные в Python/SymPy, оказались полностью идентичны результатам, полученным при ручной и полуавтоматической обработке. Графическое сравнение волновых профилей при идентичных наборах параметров (σ , C_0 , C_1 , b , a) также продемонстрировало полное совпадение, что исключает наличие скрытых ошибок в логике подстановки, дифференцирования и упрощения выражений.

Оценка вычислительной сложности показывает, что основную вычислительную нагрузку несёт этап решения нелинейной алгебраической системы, возникающей после подстановки анзаца в исходное ОДУ. Благодаря использованию оптимизированных процедур библиотеки SymPy и стратегии поэтапного исключения неизвестных, время символьного вычисления для уравнения Курамото–Сивашинского составляет доли секунды на стандартном оборудовании. Алгоритм демонстрирует высокую устойчивость к вырожденным случаям (например, при $b = 0$ или $\sigma = 0$), корректно обрабатывая их через механизм проверки ранга системы и автоматической фильтрации тривиальных решений.

Ключевым преимуществом предложенного подхода является полное исключение ручных арифметических операций, которые традиционно являются основным источником ошибок при работе с полиномами высокой степени и смешанными производными. Кроме того, реализация на языке Python с открытым исходным кодом обеспечивает готовность инструментария к интеграции в общие вычислительные пайплайны анализа нелинейных дифференциальных уравнений.

3. Поиск порядка сингулярности

3.1 Введение в проблему сингулярностей

Анализ поведения решений дифференциальных уравнений в окрестности особых точек является фундаментальным этапом при изучении интегрируемости систем и поиске точных решений. Понимание характера особенности позволяет предсказать структуру решения и выбрать подходящий метод анализа.

3.1.1. Определение сингулярной точки решения ОДУ

Рассмотрим автономное обыкновенное дифференциальное уравнение (ОДУ) порядка n в неявном виде:

$$F(y, y', y'', \dots, y^{(n)}) = 0, \quad (30)$$

где $y = y(z)$ — искомая функция, а z — независимая комплексная переменная.

Точка $z_0 \in \mathbb{C}$ называется сингулярной точкой решения $y(z)$, если в этой точке функция $y(z)$ не является аналитической (голоморфной). В окрестности такой точки решение не может быть представлено в виде степенного ряда Тейлора.

Для исследования локального поведения решения в окрестности z_0 вводится малый параметр:

$$\xi = z - z_0. \quad (31)$$

В этом случае анализ поведения решения при $z \rightarrow z_0$ сводится к анализу предельного перехода $\xi \rightarrow 0^+$. Нас интересует асимптотика решения в виде $y(z) \sim A\xi^{-p}$ при $\xi \rightarrow 0$.

3.1.2 Классификация особенностей

Характер сингулярности определяется ведущим показателем степени p в асимптотическом представлении решения $y(z) \sim A\xi^{-p}$. На основе значения p и типа асимптотики выделяют следующие классы особенностей:

- Полус. Функция имеет полюс порядка p , если

$$y(z) \sim \frac{A}{(z - z_0)^p},$$

где $A \neq 0$ и порядок p является строго положительным целым числом ($p \in \mathbb{Z}_{>0}$). В коде анализа такие точки классифицируются как `kind='pole'`.

- Алгебраическая точка ветвления. Если показатель степени p является положительным рациональным, но не целым числом ($p \in \mathbb{Q} \setminus \mathbb{Z}$, $p > 0$), то в точке z_0 решение имеет точку ветвления. Это означает многозначность решения в комплексной плоскости. В кодовой реализации соответствует `kind='branch_point'`.
- Логарифмическая особенность. Возникает, если уравнение содержит экспоненциальную нелинейность вида $e^{\alpha y}$. В этом случае стандартный степенной анзац неприменим, и решение ведет себя как $y(z) \sim -p \ln \xi$. Классифицируется как `kind='logarithmic'`.
- Существенная особенность. Возникает в уравнениях, содержащих осциллирующие или гиперболические функции от y (например, $\sin y$, $\cos y$, $\sinh y$). В таких случаях решение не имеет степенного характера (ряда Лорана не существует). Код возвращает `kind='essential'`.

- Нуль (Zero). Случай, когда показатель $p < 0$. Функция стремится к нулю в точке z_0 .

3.1.3 Польза информации о порядке полюса

Определение характера особенности не является самоцелью; оно играет ключевую роль в качественной теории дифференциальных уравнений по следующим причинам:

1. Проверка свойства Пенлеве. ОДУ обладает свойством Пенлеве, если все его подвижные особые точки являются полюсами (т.е. $p \in \mathbb{Z}_{>0}$). Это свойство является необходимым условием интегрируемости уравнения. Если метод доминантного баланса выдает дробное p или указывает на наличие логарифмических ветвей, это свидетельствует о нарушении свойства Пенлеве и, как правило, об отсутствии решения в терминах классических функций (эллиптических или рациональных).
2. Построение точных решений. Знание порядка полюса p позволяет использовать метод простых уравнений (метод анзацев). Если p — целое положительное число, то точное решение можно искать в виде конечной суммы по степеням вспомогательной функции $Y(z)$ (например, $Y' = b - Y^2$ или эллиптической функции):

$$y(z) = A_0 + A_1 Y(z) + \dots + A_p Y^p(z).$$

Значение p , найденное алгоритмом, определяет верхний предел суммирования в анзаце.

3. Классификация моделей. Анализ особенностей позволяет разделить физические модели на классы в зависимости от устойчивости их решений. Например, наличие ветвления ($p \in \mathbb{Q}$) может указывать на наличие критических явлений или фазовых переходов в моделируемой системе.

3.2 Теоретические основы метода доминантного баланса

В данном разделе излагаются математические основы метода доминантного баланса, который является ключевым инструментом для определения характера подвижных особенностей решений нелинейных обыкновенных дифференциальных уравнений (ОДУ). Метод позволяет асимптотически исследовать поведение решения в окрестности особой точки без нахождения решения в явном виде.

3.2.1. Идея метода: баланс наиболее сингулярных слагаемых

Рассмотрим автономное ОДУ порядка n в общем виде:

$$F(y, y', y'', \dots, y^{(n)}) = 0. \quad (32)$$

Пусть $z_0 \in \mathbb{C}$ — подвижная особая точка решения $y(z)$. Введем малый параметр $\xi = z - z_0$, так что анализ поведения решения при $z \rightarrow z_0$ сводится к анализу предела $\xi \rightarrow 0^+$.

Основная идея метода заключается в том, что в окрестности особой точки различные слагаемые уравнения имеют разный порядок сингулярности (порядок роста

при $\xi \rightarrow 0$). Слагаемые, имеющие наиболее отрицательный показатель степени при ξ , называются доминирующими.

Принцип доминантного баланса: Если уравнение допускает решение с алгебраическим полюсом, то должно существовать такое значение параметра $p > 0$, при котором хотя бы два слагаемых уравнения имеют одинаковый (минимальный) показатель степени ξ , и этот показатель строго меньше показателей всех остальных слагаемых. Остальные слагаемые в этом пределе считаются субдоминантными (пренебрежимо малыми по сравнению с доминирующими).

3.2.2. Алгебраический анзац

Для поиска ведущего асимптотического поведения решения используется основной алгебраический анзац:

$$y(z) \sim A\xi^{-p}, \quad \xi \rightarrow 0^+, \quad (33)$$

где $A \neq 0$ — амплитуда, а $p > 0$ — порядок полюса. Амплитуда A явно входит в выражение: при подстановке в нелинейные члены вида y^n возникает множитель $A^n \xi^{-np}$. Хотя для определения порядка p амплитуда часто сокращается, она необходима для проверки совместимости уравнения на следующем шаге (коэффициентном анализе).

3.2.3 Вычисление показателей для производных: падающий факториал

При подстановке анзаца $y = A\xi^{-p}$ в производные k -го порядка используется правило дифференцирования степенной функции:

$$\frac{d^k}{dz^k}(A\xi^{-p}) = A \cdot (-p)(-p-1) \dots (-p-k+1) \cdot \xi^{-p-k}. \quad (34)$$

Произведение $(-p)(-p-1) \dots (-p-k+1)$ называется падающим факториалом и обозначается как $(-p)^{\downarrow k}$:

$$(-p)^{\downarrow k} := \prod_{j=0}^{k-1} (-p-j). \quad (35)$$

Таким образом, асимптотика производной имеет вид:

$$y^{(k)} \sim A \cdot (-p)^{\downarrow k} \cdot \xi^{-(p+k)}. \quad (36)$$

Важно отметить, что ведущий показатель степени ξ для производной k -го порядка всегда равен $-(p+k)$, независимо от знака коэффициента $(-p)^{\downarrow k}$ (при условии, что он не обращается в ноль).

3.2.4 Нелинейные и смешанные мономы: аддитивность показателей

Для нелинейных членов, представляющих собой мономы вида $M = y^a \cdot (y')^b \cdot (y'')^c \dots$, показатель степени ξ вычисляется аддитивно. Подставляя асимптотики для y и её производных, получаем:

$$M \sim (A\xi^{-p})^a \cdot (A(-p)^{\downarrow 1} \xi^{-p-1})^b \cdot (A(-p)^{\downarrow 2} \xi^{-p-2})^c \dots \quad (37)$$

Показатель степени ξ для монома M равен:

$$E_M(p) = -ap - b(p+1) - c(p+2) - \dots \quad (38)$$

3.2.5 Рациональная нелинейность $P(y)/Q(y)$

Если уравнение содержит дробно-рациональную нелинейность вида $R(y) = \frac{P(y)}{Q(y)}$, где P и Q — полиномы от y , то при $y \rightarrow \infty$ (что соответствует $\xi \rightarrow 0$) поведение функции определяется старшими степенями числителя и знаменателя. Пусть $\deg_y P = m$ и $\deg_y Q = n$. Тогда:

$$\frac{P(y)}{Q(y)} \sim C \cdot y^{m-n} \sim C \cdot A^{m-n} \xi^{-(m-n)p}. \quad (39)$$

Ведущий показатель степени ξ равен $-(m-n)p$.

Важное замечание: Если ведущий член рациональной нелинейности имеет положительный показатель (например, $y/(1+y^2) \sim y^{-1} \sim \xi^p$ при $p > 0$), то такое слабое стремится к нулю при $\xi \rightarrow 0$ и является субдоминантным по отношению к полюсам.

3.2.6 Логарифмический анзац для уравнений с $e^{\alpha y}$

Если уравнение содержит экспоненциальную нелинейность вида $e^{\alpha y}$, алгебраический анзац $y \sim A\xi^{-p}$ приводит к существенной особенности вида $e^{\alpha A\xi^{-p}}$, для которой невозможно построить баланс степеней ξ . В этом случае применяется логарифмический анзац:

$$y(z) \sim -\frac{p}{\alpha} \ln \xi, \quad \xi \rightarrow 0^+. \quad (40)$$

Ключевое свойство этого анзаца заключается в том, что он преобразует экспоненту обратно в степенную функцию:

$$e^{\alpha y} \sim e^{\alpha(-\frac{p}{\alpha} \ln \xi)} = e^{-p \ln \xi} = \xi^{-p}. \quad (41)$$

Производные логарифмического анзаца вычисляются следующим образом:

$$\begin{aligned} y' &\sim -\frac{p}{\alpha} \xi^{-1}, \\ y'' &\sim \frac{p}{\alpha} \xi^{-2}, \\ y^{(k)} &\sim (-1)^k (k-1)! \frac{p}{\alpha} \xi^{-k}. \end{aligned}$$

Это позволяет построить баланс, например, между $y' \sim \xi^{-1}$ и $e^y \sim \xi^{-p}$, откуда следует $p = 1$.

3.2.7 Обработка трансцендентных функций

Поведение трансцендентных функций $f(y)$ при $y \rightarrow \infty$ ($\xi \rightarrow 0$) определяет их вклад в доминантный баланс:

- Ограниченные функции: Функции типа $\tanh(y)$, $\arctan(y)$, $\operatorname{sgn}(y)$ ограничены при $|y| \rightarrow \infty$.

$$\tanh(A\xi^{-p}) \rightarrow \operatorname{sgn}(A) \quad (\text{константа}). \quad (42)$$

В алгоритме такие функции заменяются на константу 1 (или -1), что соответствует показателю степени ξ^0 .

- Осцилляторные функции: Функции $\sin(y)$ и $\cos(y)$ при $y \rightarrow \infty$ осциллируют бесконечно быстро. Для них не существует разложения в ряд Лорана (степенного ряда по ξ). В классическом смысле алгебраического полюса такие уравнения не имеют, и алгоритм классифицирует их как имеющие существенную особенность (essential).
- Гиперболические функции: Функции $\sinh(y)$ и $\cosh(y)$ ведут себя экспоненциально ($\sim e^{|y|}/2$). Они также приводят к существенным особенностям, если не используется логарифмический анзац, применимый к экспонентам.

3.3 Алгоритмическая реализация (разбор кода `find_pole_order.py`)

В данном разделе описывается программная реализация метода доминантного баланса, реализованная в модуле `find_pole_order.py`. Алгоритм преобразует теоретические выкладки в последовательность операций над символьными выражениями библиотеки SymPy.

3.3.1 Низкоуровневые операции с показателями ξ

Для реализации метода баланса необходимо извлекать степени малого параметра ξ из выражений.

- `_xi_exp_of_monomial`: Функция выполняет обход дерева выражения (`preorder_traversal`) в поисках узла вида ξ^e . Возвращает показатель степени e . Используется оператор сравнения `==` вместо `is` для корректной работы с пересозданными объектами SymPy.
- `_dominant_exp`: Находит минимальный (наиболее отрицательный) показатель степени ξ в сумме мономов. Это соответствует доминирующему члену при $\xi \rightarrow 0$.
- `leading_power`: Обрабатывает дробно-рациональные выражения. Вычисляет разность минимальных показателей числителя и знаменателя ($e_{\text{num}} - e_{\text{den}}$), пропуская члены с существенными особенностями.

3.3.2 Классификация трансцендентных функций

Прежде чем применять степенной анзац, алгоритм анализирует структуру уравнения.

- `_classify_transc`: Определяет наличие функций от y , требующих особого обращения: экспонент ($e^{\alpha y}$), ограниченных функций ($\tanh$, $\arctan$), осциллирующих ($\sin$, $\cos$) и гиперболических ($\sinh$).
- `_replace_bounded`: Реализует замену $\tanh(f(y)) \rightarrow 1$, $\arctan(f(y)) \rightarrow 1$ и т.д. Поскольку в окрестности полюса $y \rightarrow \infty$, эти функции стремятся к константам, и их вклад в сингулярность определяется константой (ξ^0).

3.3.3.Обработка переменных коэффициентов

Функция `_substitute_z0_in_coefficients` отвечает за анализ уравнений с коэффициентами, зависящими от z (например, $z \cdot y'' = y^2$).

1. Алгоритм разделяет слагаемые на зависимые от y и коэффициенты.

2. Коэффициенты подставляются в точке особенности z_0 с использованием предела (`sp.limit`).
3. Если предел бесконечен, возвращается тип особенности `fixed_singularity`. Если конечен — коэффициент заменяется числом.

3.3.4. Подстановка анзаца

Центральная часть алгоритма, где производится замена y и его производных на асимптотики.

- `_term_exponents_algebraic`: Реализует подстановку $y \sim A\xi^{-p}$. Производные $y^{(k)}$ заменяются с использованием падающего факториала (функция `falling`):

$$y^{(k)} \sim A \cdot (-p)^{\downarrow k} \cdot \xi^{-p-k}, \quad (43)$$

где $(-p)^{\downarrow k} = (-p)(-p-1)\dots(-p-k+1)$. После подстановки извлекаются показатели ξ для каждого слагаемого.

- `_term_exponents_log`: Реализует логарифмический анзац $y \sim -p \ln \xi$ для уравнений, содержащих экспоненты. В этом случае производные имеют вид $y^{(k)} \sim (-1)^k (k-1)! p \xi^{-k}$, а экспонента становится степенной функцией $e^{\alpha y} \sim \xi^{-\alpha p}$.

3.3.5. Баланс показателей

- `_balance_single`: Реализует поиск p для одного уравнения.
 1. Попарный баланс: Приравниваются показатели $e_i(p) = e_j(p)$ для всех пар слагаемых.
 2. Валидация: Кандидат p^* считается валидным, если при его подстановке минимальный показатель достигается хотя бы у двух слагаемых (принцип доминантного баланса).
 3. Правило максимального p : Из всех валидных кандидатов выбирается максимальное значение, соответствующее наиболее сингулярному балансу (согласно свойству Пенлеве).
- `_balance_system`: Расширение метода для систем уравнений. Строится система алгебраических уравнений на вектор показателей $\mathbf{p} = (p_1, \dots, p_N)$ и решается символично.

3.3.6. Классификация результата

Функция `classify_p` преобразует найденное значение p в строковый идентификатор `kind`, описывающий тип особенности:

- `pole`: $p \in \mathbb{Z}_{>0}$ (настоящий полюс).
- `branch_point`: $p \in \mathbb{Q} \setminus \mathbb{Z}, p > 0$ (точка ветвления).
- `zero`: $p < 0$ (функция стремится к нулю).
- `logarithmic`: найден через логарифмический анзац.
- `essential`: для уравнений с $\sin(y)$, $\cos(y)$ и т.д.

3.3.7 Публичный API

Пользовательский интерфейс модуля представлен двумя функциями:

- `find_pole_order(eq, y_func, z)`: Анализирует одно ОДУ. Принимает уравнение, неизвестную функцию и независимую переменную. Возвращает кортеж (p, kind) .
- `find_pole_order_system(equations, y_funcs, z)`: Анализирует систему ОДУ. Возвращает словарь $\{y_i : p_i\}$ и общий тип особенности системы.

3.4 Верификация и тестирование

Для подтверждения корректности и надежности разработанного алгоритма была создана система модульного тестирования (файл `test_pole_order.py`). Тестирование проводилось на широком спектре дифференциальных уравнений, охватывающем как классические интегрируемые случаи, так и уравнения с особенностями, отличными от полюсов.

3.4.1 Набор тестов

Общий объем тестового набора составляет 82 тестовых сценария, разделенных на следующие категории:

- Одиночные уравнения (78 шт.):
 - Уравнения Пенлеве I–IV (проверка свойства Пенлеве).
 - Уравнения математической физики: КдФ, Бюргерса, НЛШ, Курамото–Сивашинского, Фишера, Чейзи.
 - Специфические случаи: дробные степени, рациональная нелинейность, смешанные мономы, экспоненциальные функции, логарифмические поправки.
 - Системы с переменными коэффициентами и диссипацией.
- Системы уравнений (4 шт.):
 - Системы вида $y' = w, w' = f(y)$ для проверки векторного анзаца $y_j \sim A_j \xi^{-p_j}$.

В результате прогона всех тестов алгоритм показал 100% точность совпадения с теоретическими ожиданиями.

3.4.2 Критерии успеха

Автоматическая проверка решения уравнения на корректность осуществляется путем сравнения кортежа результатов $(p_{\text{расч}}, \text{kind}_{\text{расч}})$ с эталонными значениями $(p_{\text{этал}}, \text{kind}_{\text{этал}})$, заданными в коде.

$$\text{Success} \iff (p_{\text{расч}} == p_{\text{этал}}) \wedge (\text{kind}_{\text{расч}} == \text{kind}_{\text{этал}}). \quad (44)$$

Здесь p — порядок особенности, а kind — классификационный тег (например, `'pole'`, `'branch_point'`, `'zero'`).

3.4.3 Граничные случаи

Алгоритм корректно обрабатывает специфические ситуации, требующие отдельной логики:

1. Вырожденные балансы: В случаях, когда баланс невозможен (например, $y'' = (y')^2$, где старшая производная не балансирует с нелинейностью), алгоритм возвращает `kind='unknown'`.
2. Фиксированные особенности: При наличии переменных коэффициентов, стремящихся к бесконечности в точке z_0 (например, $\frac{1}{z}y''$ при $z \rightarrow 0$), детектируется `kind='fixed_singularity'`.
3. Отсутствие полюса (Нули): Для уравнений типа $y' = 1/y$ решение стремится к нулю при $z \rightarrow z_0$. При включенном флаге `allow_nonpositive` алгоритм корректно определяет отрицательный порядок $p < 0$ и классифицирует его как `kind='zero'`.

3.5 Ограничения метода и направления развития

Несмотря на высокую эффективность, метод доминантного баланса имеет определенные ограничения, связанные с природой анализа степенных рядов.

3.5.1 Ограничения метода

Метод не применим или требует модификации в следующих случаях:

- Существенные особенности: Уравнения, содержащие осциллирующие функции ($\sin y, \cos y$) или гиперболические функции ($\sinh$), не допускают описания решения в виде степенного ряда или логарифма из-за бесконечных осцилляций при $\xi \rightarrow 0$. В таких случаях алгоритм возвращает статус `'essential'`, что сигнализирует о невозможности алгебраического баланса.
- Отсутствие алгебраического баланса: Для некоторых нелинейных комбинаций (например, $y'' = (y')^2$) невозможно подобрать $p > 0$, при котором старшая производная и нелинейный член уравновешивают друг друга.

3.5.2 Возможные расширения

1. Учет логарифмических поправок: Текущая реализация определяет только логарифмический характер особенности (`'logarithmic'`). Перспективным направлением является автоматическое нахождение коэффициента перед логарифмом в анзаце $y \sim -p \ln \xi$.
2. Автоматический поиск амплитуды A : В данный момент алгоритм находит только показатель степени p . Интеграция шага подстановки $y \sim A\xi^{-p}$ обратно в уравнение позволит автоматически вычислять коэффициент A (амплитуду полюса), что необходимо для полного построения главного члена асимптотики.

3.5.3 Интеграция с другими методами

Разработанный модуль `find_pole_order` является фундаментом для более сложных методов построения точных решений:

- Метод простых уравнений (Риккати, Вейерштрасс): Значение p , найденное данным алгоритмом, напрямую определяет степень полинома в анзаце $y(z) = \sum_{k=0}^p A_k Y^k(z)$. Автоматическое определение p позволяет динамически формировать структуру анзаца без ручного перебора.
- Свойство Пенлеве: Модуль может служить автоматическим тестом Пенлеве: если для всех подвижных особенностей уравнения `kind` является `'pole'` (и $p \in \mathbb{Z}_{>0}$), уравнение с высокой вероятностью интегрируемо.

3.5.4 Краткое резюме

Разработанный инструмент предоставляет надежный и быстрый способ анализа сингулярной структуры решений нелинейных ОДУ. Он успешно определяет порядок полюса, классифицирует тип особенности (полюс, точка ветвления, логарифмическая особенность) и устойчиво работает со сложными классами уравнений (дробные степени, системы, переменные коэффициенты). Интеграция данного модуля в вычислительный конвейер (вместе с решателями уравнений Курамото–Сивашинского) позволяет существенно автоматизировать процесс поиска точных аналитических решений, минимизируя влияние человеческого фактора при рутинных алгебраических преобразованиях.

4. Заключение

В ходе выполнения научно-исследовательской работы была успешно решена поставленная задача по автоматизации аналитического исследования нелинейных дифференциальных уравнений. Разработанный вычислительный комплекс объединяет два независимых программных модуля: универсальный анализатор сингулярностей на базе метода доминантного баланса и символический решатель уравнения Курамото–Сивашинского, использующий анзацы Риккати и эллиптической функции Вейерштрасса.

Основные результаты работы заключаются в следующем:

1. Реализован и верифицирован алгоритм поиска порядка подвижной особенности для одиночных ОДУ и систем уравнений. Алгоритм корректно обрабатывает полиномиальные, рациональные, дробные и трансцендентные нелинейности, а также уравнения с переменными коэффициентами. Тестирование на наборе из 82 классических уравнений математической физики (уравнения Пенлеве I–IV, Кортевега–де Фриза, Бюргерса, Курамото–Сивашинского, Абеля и др.) показало 100% совпадение с известными аналитическими эталонами.
2. Создан модуль поиска точных решений уравнения Курамото–Сивашинского. Автоматизирована процедура подстановки анзацев, приведения уравнения к системе алгебраических уравнений и нахождения неизвестных коэффициентов. Получено 10 семейств решений в базисе уравнения Риккати и 2 семейства решений в базисе функции Вейерштрасса.
3. Проведена классификация полученных решений в зависимости от знака параметра b в определяющем уравнении: при $b > 0$ формируются солитонные и кинк-профили, при $b < 0$ — периодические и полюсные структуры.
4. Выполнен переход от прототипирования в коммерческой системе Maple к открытой реализации на языке Python с использованием библиотеки SymPy. Это обеспечило кроссплатформенность, прозрачность алгоритмов и возможность интеграции модулей в современные научные вычислительные среды.

Практическая значимость работы обусловлена созданием готового к применению инструментария, который устраняет необходимость ручных громоздких символьных выкладок, минимизирует риск арифметических ошибок и существенно сокращает время поиска аналитических решений. Разработанные модули могут использоваться как самостоятельные исследовательские инструменты, так и в составе более крупных аналитических конвейеров для уравнений математической физики.

Перспективы дальнейших исследований связаны с расширением функционала комплекса: поддержкой произвольных систем уравнений высших порядков, добавлением процедур вычисления амплитуды полюса A на следующем шаге асимптотического разложения, а также интеграцией с численными методами для верификации аналитических профилей. Кроме того, планируется разработка пользовательского интерфейса и экспорт результатов в форматы, совместимые с популярными системами компьютерной алгебры.

Таким образом, цели и задачи научно-исследовательской работы достигнуты в полном объёме, а полученные результаты подтверждают работоспособность предложенных алгоритмов и их готовность к применению в задачах нелинейной математической физики.

5. Список литературы и интернет-ресурсов

- [1] Н. А. Кудряшов. *Методы нелинейной математической физики*, 2-е изд., доп. — Долгопрудный: Издательский дом «Интеллект», 2010. — 368 с.
- [2] А. Д. Полианин, В. Ф. Зайцев. *Справочник по нелинейным дифференциальным уравнениям*. — М.: Физматлит, 2002.
- [3] R. Conte, M. Musette. *The Painlevé Handbook*. — Springer, 2008.
- [4] Y. Kuramoto. *Chemical Oscillations, Waves, and Turbulence*. — Springer, 1984.
- [5] N. A. Kudryashov. Exact solutions of the generalized Kuramoto–Sivashinsky equation. *Physics Letters A*, 1990, Vol. 147, P. 287–291.
- [6] Н. А. Кудряшов. Простейшие уравнения для поиска точных решений нелинейных дифференциальных уравнений. *Прикладная математика и механика*, 2005, Т. 69, № 1, С. 34–48.
- [7] <https://docs.sympy.org/> — Официальная документация библиотеки SymPy.
- [8] <https://www.python.org/> — Официальный сайт языка Python.
- [9] <https://matplotlib.org/> — Библиотека визуализации данных Matplotlib для Python.
- [10] <https://github.com/Korabl3va/NIR> — Репозиторий с исходным кодом и материалами научно-исследовательской работы.

Приложение А. Доступ к исходным материалам

Полный исходный код программ, и дополнительные данные, необходимые для воспроизведения результатов работы, размещены в открытом репозитории.

Все необходимые материалы доступны по ссылке:

<https://github.com/Korabl3va/NIR>